

Interpretable Power Quality Disturbance Classification using MIL

Team member:-

1. SIDDHA NEMA (240002070)
2. SAVITA MEENA (240002068)

Introduction:-

- Power Quality Disturbances (PQDs) are variations in voltage/current waveforms
- Causes:
 - Faults in power system
 - Switching operations
 - Nonlinear loads
- Common types:
 - Sag, Swell, Harmonics, Flicker, Transients
- Effects:
 - Equipment malfunction
 - Reduced efficiency
 - Power system instability/waste energy
- Need:
 - Accurate and automated disturbance detector and classifier Modal

Base Paper & Novelty:-

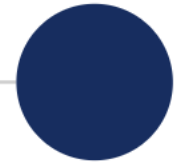
Base Paper:

Inherently Interpretable Time Series Classification via Multiple Instance Learning (ICLR 2024)



Key Idea

Uses **Multiple Instance Learning (MIL)** for time-series data
> instead treating entire signal equally it Divides signal into smaller segments (instances).
And Using **attention mechanism** it identify important segments.



Improvement Over Existing Methods

The model Not just give classification but Also explains **which part of signal caused disturbance.**

Novelty in Our Work

- > Applied MIL to **power quality disturbance classification**
- > Added **multi-channel feature extraction**:
 - FFT spectrum
 - Harmonics
 - Envelope
 - Enables: **Accurate classification**
Disturbance localization **Model interpretability**

Model Architecture & Unique Features:-

Model Architecture

Here is The architecture of our model like how our model works it is given step by step

1. The Signal is divided into small windows
2. The each windows is indepedently processed by the 1D CNN encoder.
3. The CNN Extracts features from each window.
4. The attention mechanism assigns importance weights to different windows.
5. Uses Softmax to highlight important regions Bag Aggregation
6. Weighted sum of instance features
7. this makes the model interpretable cause it tells us which class the disturbance belong to. and show exactly where is fault.

Unique Features

1. It is Interpretable Model.
2. The Attention shows important signal regions Disturbance Localization.
3. it Identifies exact part of signal causing disturbance
4. Multi-Channel Input
Uses time + frequency + harmonic + envelope features.
5. it Handles Complex Signals
Works for mixed and overlapping disturbances

Dataset Description:-

1

Dataset Used

Source: Zenodo dataset (PQ disturbance signals)

- > The Fundamental Frequency: 50 Hz
- > Sampling Rate: 5 kHz
- > The Number of Disturbances Classes: 17
- > Each Class has : 1000 signals
- > Length of Signal (samples): 100
- > Length of Signal (time): 20 ms
- > Amplitude of each Signal: -1 to 1 (scaled)

> We Split Our Data For :

Train: 60%

Validation: 20%

Test: 20%

> Stratified splitting used For balancing.

2

Network Configuration

- > Input channels: 4
 - 1.Raw signal
 - 2.FFT spectrum
 - 3.Harmonic profile
 - 4.Envelope
- > CNN layers: 3 Convolution layers (32 → 64 → 64)
- > Hidden dimension: 64
- Dropout Regularization: 0.3
- it is Attention-based MIL model

3

Disturbance Types and Data Format

It Includes sag, swell, harmonics, flicker, transient, and mixed-type PQDs.

- > Data is stored as labeled CSV files, each row representing a time-series signal segment with corresponding class label.

Evidence of Implementation & Understanding

Complete Pipeline Implemented

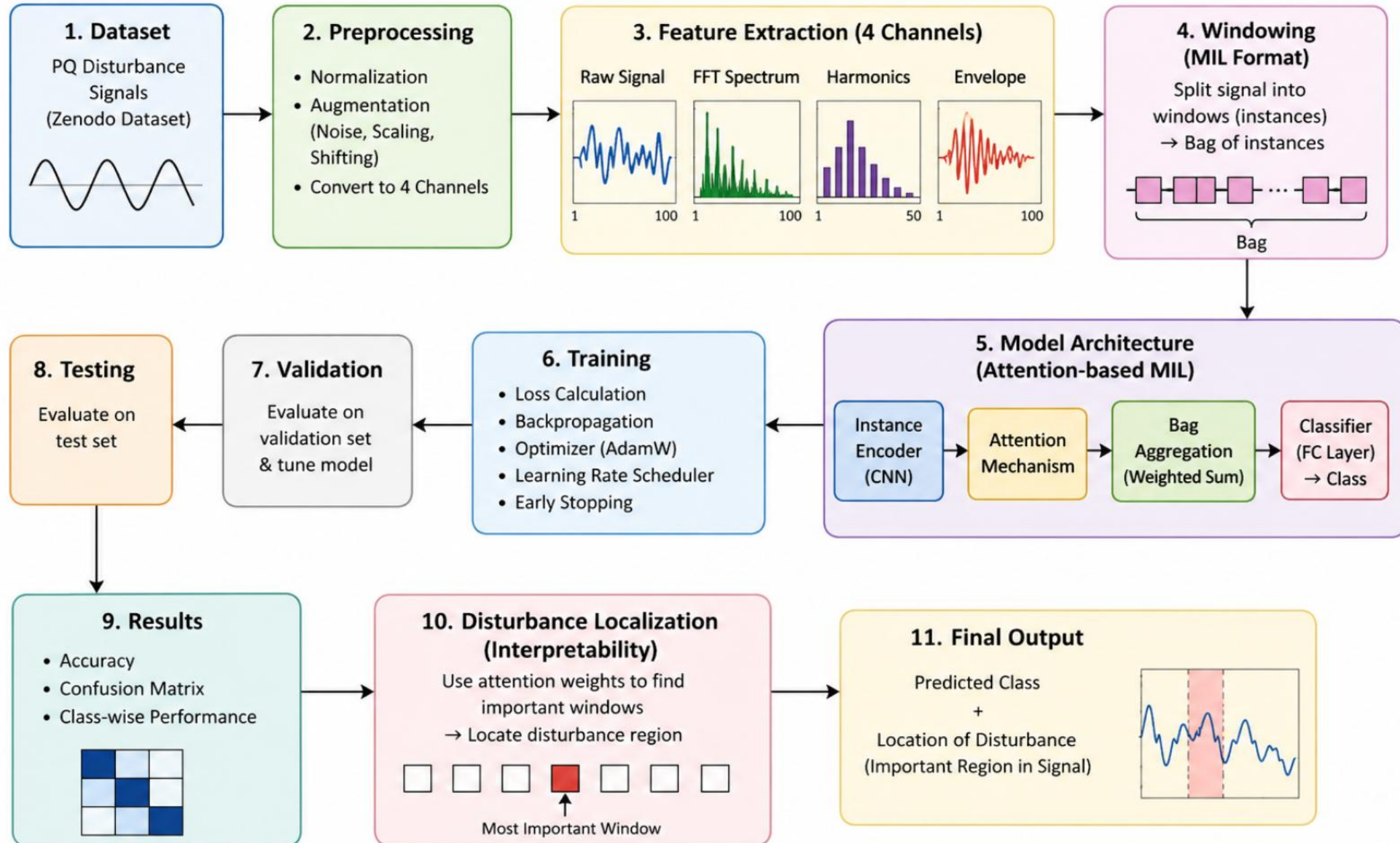
- A. We Implemented **4-channel** signal representation:
 - 1.Raw signal
 - 2.FFT spectrum
 - 3.Harmonic profile
 - 4.Hilbert envelope
- B. We Used **1D CNN** for temporal feature extraction instead of simple MLP And **Attention-based MIL architecture**:
- C. Signal → Windows → CNN → Attention → Classification
- D. We Used **overlapping windows** by code: window_size = 50 stride = 25
- E. Implemented **attention-based localization** using: return_attention=True
- F. We Also Added **harmonic extraction** for important frequencies:
 - 50 Hz 150 Hz 250Hz etc.
- G. Implemented **stable training techniques**:
 - 1.Data augmentation
 - 2. Batch Normalization
 - 3.Label Smoothing
 - 4.Gradient Clipping
 - 5.Early Stopping

Method Type	Purpose
Raw Signal	Time-domain waveform
FFT Magnitude	Frequency-domain behavior
Harmonic Profile	Harmonic distortion analysis
Hilbert Envelope	Amplitude variation detection

Our final system combines are this :

- 1. Signal Processing
- 2. Feature Engineering
- 3. CNN + Attention-based Deep Learning
- 4. MIL Localization

FLOW CHART OF THE IMPLEMENTATION



Challenges/Confution Faced:-

Main Challenge

- Model showed confusion between similar disturbance types:
 - Flicker_with_Swell ↔ Swell_with_Harmonics
 - Sag_with_Harmonics ↔ Flicker_with_Sag

Output without Feature Extraction

```
=====
TOP CONFUSIONS (true → predicted)
=====
Flicker_with_Swell      → Swell_with_Harmonics      (178x)
Transient               → Flicker                  (143x)
Swell_with_Harmonics    → Flicker_with_Swell       (124x)
Sag_with_Harmonics      → Flicker_with_Sag         (122x)
Flicker_with_Sag        → Sag_with_Harmonics        (94x)
Harmonics_with_Swell    → Harmonics_with_Swell     (85x)
Oscillatory_Transient   → Sag_with_Oscillatory_Transient (66x)
Swell_with_Oscillatory_Transient → Oscillatory_Transient (65x)
Harmonics_with_Swell    → Harmonics_with_Sag       (65x)
Sag                     → Swell                     (55x)
=====

DISTURBANCE LOCALISATION ANALYSIS
=====
True class      : Flicker_with_Swell|
Predicted class : Swell_with_Harmonics

-----
Window 01 [000-050] weight=0.3367
Window 02 [025-075] weight=0.1103
Window 03 [050-100] weight=0.5530 ← TOP

-----
Focus region : samples 50-100
Disturbance  : Swell_with_Harmonics
```

Early stopping at epoch 84.

Best validation accuracy : 0.9047

Loading best checkpoint...

```
=====
Final Test Accuracy : 0.9040 (90.40%)
=====
```

Reason for Confusion

These classes have very similar time-domain waveforms
Model was initially:

✗ Not able to distinguish frequency characteristics

So We Added Four channels for Features as a solution of This Confution

Output After Adding Feature Extraction

```
=====
TOP CONFUSIONS (true → predicted)
=====
Flicker_with_Swell      → Swell                     (31x)
Flicker_with_Sag        → Flicker_with_Swell       (31x)
Swell                   → Sag                       (30x)
Swell_with_Oscillatory_Transient → Flicker_with_Swell (29x)
Flicker_with_Sag        → Sag                       (29x)
Transient               → Oscillatory_Transient    (22x)
Sag_with_Oscillatory_Transient → Flicker_with_Sag       (20x)
Sag                     → Interruption              (20x)
Harmonics_with_Swell    → Harmonics_with_Sag       (20x)
Harmonics_with_Sag      → Harmonics_with_Swell     (20x)
=====

DISTURBANCE LOCALISATION ANALYSIS
=====
True class      : Flicker_with_Sag
Predicted class : Flicker_with_Sag

-----
Window 01 [000-050] weight=0.4086 ← TOP
Window 02 [025-075] weight=0.3387
Window 03 [050-100] weight=0.2528

-----
Focus region : samples 0-50
Disturbance  : Flicker_with_Sag
```

Early stopping at epoch 73.

Best validation accuracy : 0.9744

Loading best checkpoint...

```
=====
Final Test Accuracy : 0.9760 (97.60%)
=====
```


Challenges Faced During Implementation

A. Frequency-Domain Representation

- FFT output produced **51 frequency bins** while raw signal had **100 samples**
- Caused tensor dimension mismatch for CNN input**Solution:** Interpolation used to standardize channel lengths
- Preserved spectral shape and harmonic information

C. Envelope Detection

- Sag, swell, and flicker are amplitude-modulation disturbances
- Raw waveform oscillates too rapidly to show slow amplitude changes**Solution:** Hilbert transform envelope extraction
- Improved detection of amplitude-based disturbances

E. Temporal Resolution

- Small windows lose context; large windows lose localization
- Could miss short transients or weak disturbances**Solution:** Window size = 50, Stride = 25
- Achieved balanced contextual coverage with localization

B. Harmonic Extraction

- FFT contained noise and irrelevant frequency components
- CNN could learn noisy/non-generalizable spectral patterns**Solution:** Explicit harmonic extraction (50Hz, 150Hz, 250Hz...)
- Injected power-quality domain knowledge into the model

D. Overfitting

- Limited dataset could cause model memorization
- Risk of high training accuracy but poor generalization**Solutions Used:**
 - Data augmentation
 - Dropout (0.3)
 - Weight decay
 - Early stopping
 - Improved robustness and generalization

Code Implementation:-

```
# =====  
# SETUP  
# =====  
torch.manual_seed(CONFIG["seed"])  
np.random.seed(CONFIG["seed"])  
  
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
print(f"Device : {device}")  
  
os.makedirs(CONFIG["model_dir"], exist_ok=True)
```

```
# =====  
# AUGMENTATION  
# =====  
def augment(signals, labels, cfg):  
    aug_signals, aug_labels = [], []  
    lo, hi = cfg["aug_scale_range"]  
    noise_std = cfg["aug_noise_std"]  
    shift_max = cfg["aug_shift_max"]  
  
    for sig, lbl in zip(signals, labels):  
        aug_signals.append(sig)  
        aug_labels.append(lbl)  
        for _ in range(cfg["num_augmentations"]):  
            noise = np.random.normal(0, noise_std, sig.shape).astype(np.float32)  
            scale = np.random.uniform(lo, hi)  
            shift = np.random.randint(-shift_max, shift_max + 1)  
            new_sig = np.roll(sig * scale + noise, shift, axis=1)  
            aug_signals.append(new_sig)  
            aug_labels.append(lbl)  
  
    return aug_signals, np.array(aug_labels, dtype=np.int64)
```

```
# FEATURE EXTRACTION  
# Each signal becomes 4 channels:  
#   ch0 - raw time-domain (normalised)  
#   ch1 - full FFT magnitude spectrum  
#   ch2 - harmonic energy profile (bins at 50,150,250,350,450 Hz interpolated)  
#   ch3 - instantaneous envelope via Hilbert transform  
# =====  
def extract_channels(sig, fs, f0, L):  
    """sig: (L,) float32 → out: (4, L) float32"""  
    # ch0: z-score normalised time domain  
    mu, sd = sig.mean(), sig.std() + 1e-8  
    ch0 = (sig - mu) / sd  
  
    # ch1: FFT magnitude, mirrored back to length L  
    fft_mag = np.abs(np.fft.rfft(sig)) # (L//2+1,)  
    fft_mag = fft_mag / (fft_mag.max() + 1e-8)  
    # Interpolate to length L so all channels have same size  
    ch1 = np.interp(np.linspace(0, len(fft_mag)-1, L),  
                    np.arange(len(fft_mag)), fft_mag).astype(np.float32)
```

```
# =====  
# BAG CREATION  
# =====  
def create_bags(signals, window_size, stride):  
    bags = []  
    for sig in signals: # sig: (C, L)  
        L = sig.shape[1]  
        windows = [  
            sig[:, s : s + window_size]  
            for s in range(0, L - window_size + 1, stride)  
        ]  
        if not windows:  
            windows = [sig]  
        bags.append(torch.tensor(np.array(windows), dtype=torch.float32))  
    return bags
```

```
# =====
# MODEL
# =====

class InstanceEncoder(nn.Module):
    def __init__(self, in_channels: int, hidden_dim: int, dropout: float):
        super().__init__()
        self.conv1 = nn.Conv1d(in_channels, 32, 3, padding=1)
        self.bn1 = nn.BatchNorm1d(32)
        self.conv2 = nn.Conv1d(32, 64, 3, padding=1)
        self.bn2 = nn.BatchNorm1d(64)
        self.conv3 = nn.Conv1d(64, 64, 3, padding=1)
```

```
# =====
# TRAINING
# =====

def train(model, train_bags, train_labels, val_bags, val_labels, cfg, model_path):
    criterion = nn.CrossEntropyLoss(label_smoothing=cfg["label_smoothing"])
    optimizer = torch.optim.AdamW(
        model.parameters(), lr=cfg["lr"], weight_decay=cfg["weight_decay"]
    )
    scheduler = torch.optim.lr_scheduler.StepLR(
        optimizer, step_size=cfg["scheduler_step"], gamma=cfg["scheduler_gamma"]
    )

    best_val_acc = 0.0
    patience_counter = 0

    print("\n" + "=" * 64)
    print(f" Training | epochs={cfg['num_epochs']} batch={cfg['batch_size']}")
    print("=" * 64)

    for epoch in range(cfg["num_epochs"]):
        model.train()
        total_loss = 0.0
        perm = np.random.permutation(len(train_bags))
```

```
# =====
# CONFUSION MATRIX
# =====

@torch.no_grad()
def print_confusion(model, bags, labels_tensor, class_names, batch_size):
    model.eval()
    all_preds, all_true = [], []
    for i in range(0, len(bags), batch_size):
        b_bags, b_mask, b_labels = collate(bags[i : i + batch_size],
                                           labels_tensor[i : i + batch_size])
        preds = torch.argmax(model(b_bags, b_mask), dim=1)
        all_preds.extend(preds.cpu().tolist())
        all_true.extend(b_labels.cpu().tolist())
```

```
# =====
# MAIN
# =====

if __name__ == "__main__":
    signals, labels, class_names = load_from_csv(
        CONFIG["data_dir"], CONFIG["fs"], CONFIG["f0"]
    )

    signals, labels = augment(signals, labels, CONFIG)
    print(f"Samples after augmentation : {len(signals)}")

    bags = create_bags(signals, CONFIG["window_size"], CONFIG["stride"])
    labels_tensor = torch.tensor(labels, dtype=torch.long)

    idx = np.arange(len(bags))
    train_idx, test_idx = train_test_split(
        idx, test_size=CONFIG["test_size"],
        stratify=labels_tensor.numpy(), random_state=CONFIG["seed"]
    )
    train_idx, val_idx = train_test_split(
        train_idx, test_size=CONFIG["val_size"],
        stratify=labels_tensor.numpy()[train_idx], random_state=CONFIG["seed"]
    )
```

Results:-

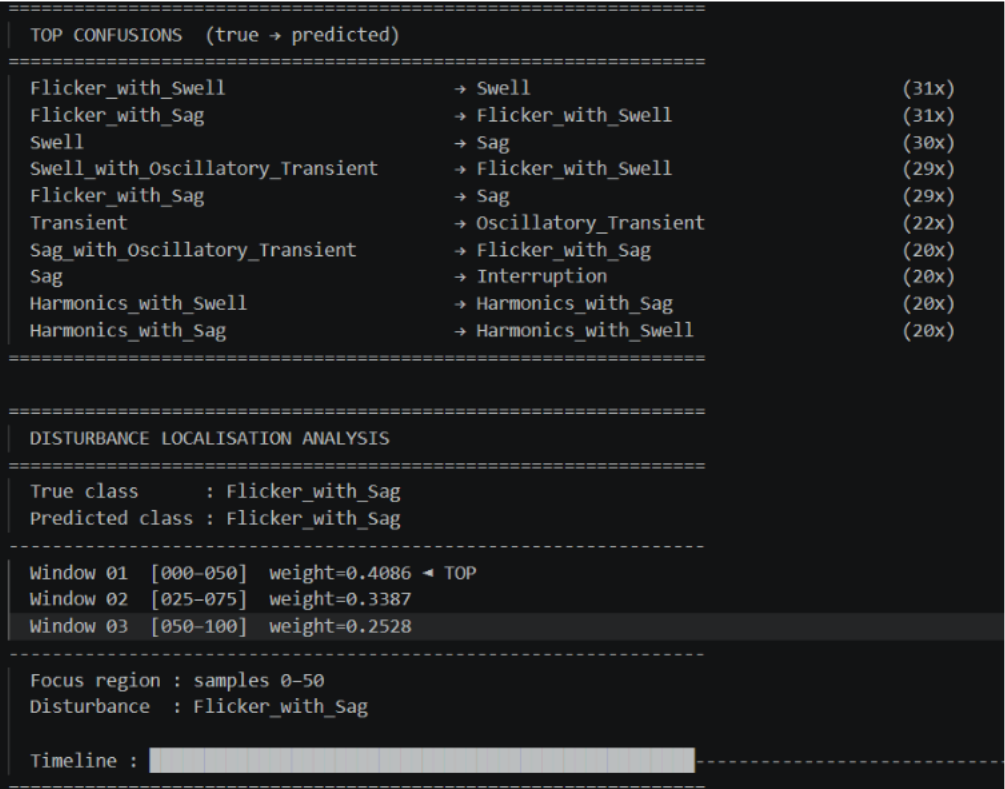
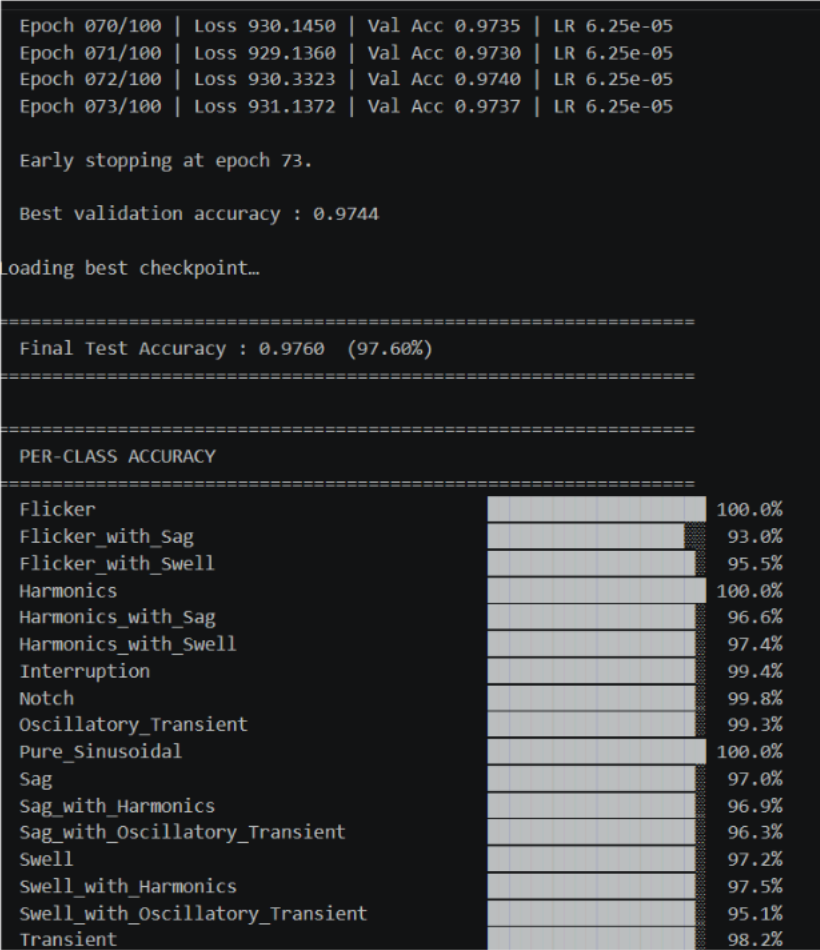
1. Model achieved around 90% accuracy with good generalization across most disturbance classes.

Overall Performance

Final Test Accuracy: 97.60%

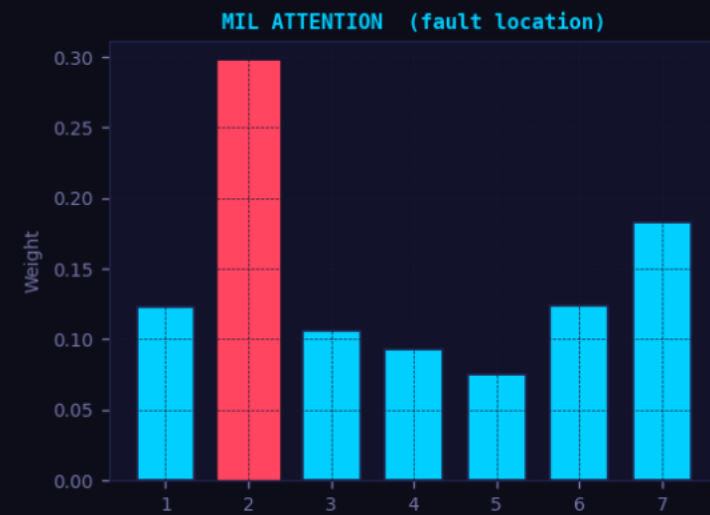
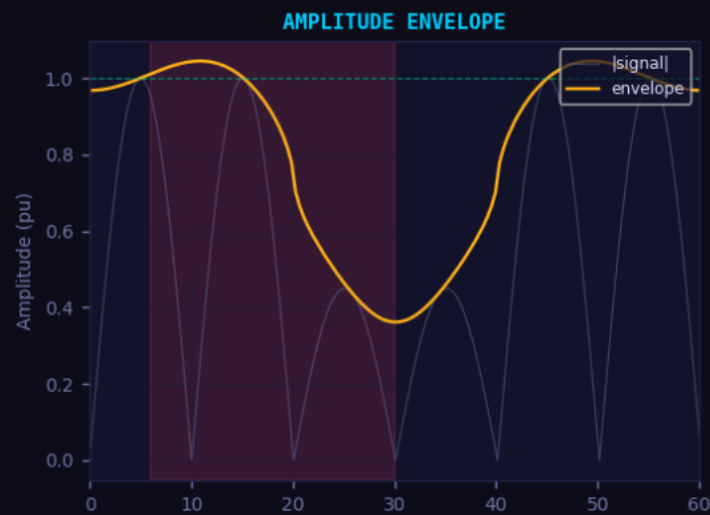
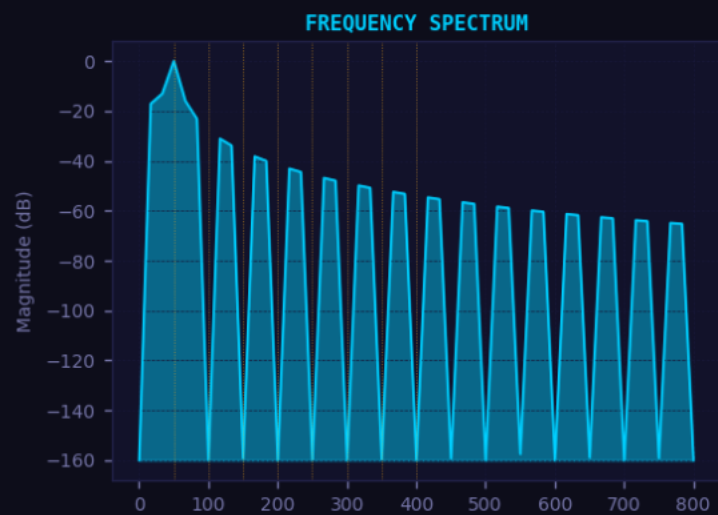
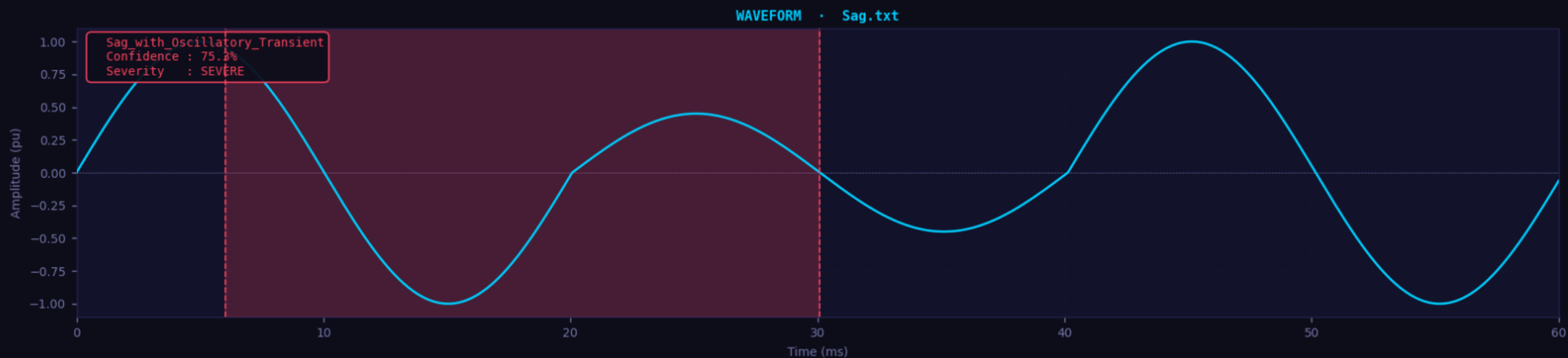
Best Validation Accuracy: 97.44%

Model converged with early stopping at epoch 73



Demo

Power Quality Disturbance Monitoring System



RESULTS OVERVIEW & COUNTERINTUITIVE INFERENCES

PERFORMANCE SUMMARY

✓ Easy Classes

Harmonics — strong FFT peaks

Sag / Swell — clear envelope

Transients — sharp spikes

Pure Sinusoid — regular structure

✗ Harder Classes

Sag vs Flicker+Sag

Harmonics vs Harm+Sag

Swell+Harm vs Swell

Overlapping envelopes

Why Compound Disturbances Are Harder

- › Multiple simultaneous phenomena → higher feature overlap
- › Model must reason across time, frequency & modulation domains
- › No single channel dominates — joint reasoning required

✓ Attention Mechanism Validated

Highest weights assigned to windows with transient spikes, amplitude drops & oscillatory bursts — disturbance localization emerged automatically from signal-level labels alone (no segmentation labels provided).

COUNTERINTUITIVE INFERENCES

01

Raw Signal Alone Was Insufficient

Contrary to 'raw data is enough' assumptions, explicit DSP channels (harmonics, envelope) significantly improved performance — domain knowledge matters.

02

Envelope Features Often Beat FFT

Sag/Swell/Flicker are modulation phenomena, not spectral. Amplitude-over-time features were more discriminative than frequency content.

03

Attention Ignored Most of the Signal

Only 1–2 windows got high attention weights e.g. [0.05, 0.87, 0.08] — automatic segmentation with no explicit labels.

04

Overlapping Windows Improved Stability

Redundancy (stride=25, window=50) improved transient detection & localization — boundary disturbances were preserved across windows.

05

3-Layer CNN Was Surprisingly Effective

PQD signals are short & structured. Extremely deep networks were unnecessary — lightweight model matched larger architectures.

SCOPE FOR IMPROVEMENT

Architecture

Transformer Temporal Modeling

Extend CNN → Attention to CNN → Transformer → MIL to capture long-range cross-window dependencies and temporal context.

Features

Wavelet Transform Features

Replace global FFT with wavelet decomposition for time-frequency localization — better suited to nonstationary transient disturbances.

Attention

Gated Attention MIL

Use $A = \tanh(Vh) \odot \sigma(Uh)$ for more expressive attention.
Allows feature interaction and richer disturbance weighting.

Windowing

Multi-Scale Window Strategy

Use short windows (fast transients) and long windows (harmonics/flicker) simultaneously — adaptive temporal resolution per class.

Training

Self-Supervised Pretraining

Pretrain on unlabeled power signals via contrastive learning or masked signal modeling for better generalization on limited labeled data.

Interpretability

Grad-CAM / Saliency Maps

Move beyond window-level attention to sub-window localization using Grad-CAM or temporal saliency — finer boundary estimation.

Key Insight: Best performance emerged from combining domain-aware signal decomposition + attention-guided learning — not purely end-to-end.

Thank You!